

Materia

Seguridad en el desarrollo de Aplicaciones

Nombre

Ricardo Porras Reyes

Matricula

2024171008

Grupo

IDSG17

Profesor

Gerardo Ramírez Villarreal

Objetivo

En esta práctica integré Grafana, Loki y Promtail a **VulnerableApp** para centralizar y visualizar los registros generados por la aplicación sin modificar su código ni su funcionamiento.

Desarrollo

Paso 1. Verificación de la solución existente

Primero ejecuté **VulnerableApp** y comprobé que Serilog continuara generando registros correctamente. Verifiqué que los eventos se almacenaran en la consola, en la carpeta **Logs** y en **Seq**, confirmando que la aplicación seguía funcionando correctamente.

Paso 2. Investigación de la infraestructura

Investigué las imágenes oficiales de Grafana, Loki y Promtail, así como los puertos y configuraciones necesarias para ejecutar los tres servicios mediante Docker Compose.

Paso 3. Integración con Docker Compose

Creé el archivo **docker-compose.yml** para levantar únicamente los contenedores de Grafana, Loki y Promtail. La aplicación VulnerableApp continuó ejecutándose localmente con Visual Studio, tal como lo solicitaba la práctica.

Paso 4. Compartición de la carpeta Logs

Configuré un **Bind Mount** para compartir la carpeta **Logs** con Promtail. Esto permitió que el contenedor leyera directamente los archivos generados por Serilog sin realizar cambios en la aplicación.

Paso 5. Configuración de Promtail

Configuré Promtail para monitorear los archivos de la carpeta **Logs**, enviarlos a Loki y asignar labels que identifican el origen de los registros, facilitando posteriormente su búsqueda.

Paso 6. Levantamiento de la infraestructura

Ejecuté el comando `docker compose up -d` y comprobé mediante `docker ps` que los contenedores de Grafana, Loki y Promtail estuvieran en ejecución.

Paso 7. Configuración de Grafana

Ingresé a Grafana y agregué Loki como fuente de datos para poder consultar los registros enviados por Promtail.

Paso 8. Generación de actividad

Ejecuté VulnerableApp y realicé diferentes acciones, como inicios de sesión, consultas y operaciones CRUD, con el fin de generar nuevos registros para su monitoreo.

Paso 9. Validación

Finalmente comprobé que los mismos eventos generados por la aplicación podían visualizarse tanto en Seq como en Grafana, verificando que la integración funcionara correctamente.

Comparación entre Seq y Grafana + Loki

Característica	Seq	Grafana + Loki
Búsqueda	Sencilla	Avanzada con LogQL
Dashboards	Básicos	Personalizables
Alertas	Limitadas	Integradas
Uso principal	Revisión de logs	Observabilidad y monitoreo

Respuestas

¿Qué modificaciones realicé en Promtail?

Configuré Promtail para monitorear la carpeta **Logs**, enviar la información a Loki y agregar labels para identificar los registros.

¿Cómo utiliza Loki las labels?

Loki utiliza las labels para organizar e identificar los registros, permitiendo realizar consultas más rápidas y específicas.

¿Qué ventajas ofrecen para LogQL?

Las labels facilitan filtrar los registros por aplicación, categoría o tipo de evento, haciendo más eficiente el análisis de la información.

Reflexión

Esta práctica me permitió comprender que es posible incorporar una plataforma de observabilidad sin modificar la aplicación existente. Gracias a esta arquitectura, el monitoreo queda separado de la lógica de negocio, lo que facilita el mantenimiento, la escalabilidad y el análisis de incidentes en un entorno de producción.